



High Performance Computing Exercise Sheet 11

HS 25
Dr. Douglas Potter

<http://www.ics.uzh.ch/>

Teaching Assistants:

Cesare Cozza, cesare.cozza@uzh.ch

Mark Eberlein, mark.eberlein@uzh.ch

Issued: 28.11.2025

Due: 05.12.2025

In this exercise sheet, we perform some basic benchmarking of the `nbody` code and make some simple changes to improve the performance.

Exercise 1 [Introduction to `PerfTools-Lite`]

Man pages introduce the `PerfTools-Lite` as follows: "PerfTools-Lite is a simplified, easy-to-use version of the Cray Performance Measurement and Analysis Tool set. PerfTools-Lite provides basic performance analysis information automatically, with a minimum of user interaction, and yet offers information useful to users wishing to explore their program's behavior further using the full CrayPat tool set...". In this exercise, we will use this tool to analyse the run of the code `nbody.cpp` which can be found on the github repository.

- log in to Eiger and load the `cray` module, and then load the `perftools-lite` module
- compile the code `nbody.cpp` using the supplied `Makefile`. Do you observe any changes in the terminal output compared to usual compilation during the previous sessions? Run the produced executable, either interactively or using the queue system.
- once finished, inspect the output and the folder(s) produced and read few details about the output structure in the manual (`man perftools-lite`)
- which parts of the code took the most CPU time? Can you make sense of why?

Commit: Push the log file or output of the run with the `perftools-lite` tool and comment briefly on its content and what you can learn about the performance of your code.

Exercise 2 [Single and Double precision]

Unload the `perftools-lite` module and switch to the GNU programming environment. Then rebuild the code using the GNU compiler.

```
1 module unload perftools-lite
2 module swap PrgEnv-cray PrgEnv-gnu
3 make clean
4 make
```

Time the code by using the `time` command in linux. How long did it take?

In class we used Intel VTune to profile the code and noticed that there were double precision floating point operations which we did not expect. After some careful analysis of the code we realized that we were calling the `sqrt` function which was converting to `double` and then converting back. Change the code to use the `sqrtdf` function, recompile and time it again. How long does it take now?

Exercise 3 [Vectorisation]

We discussed the code in class and realized that the compiler cannot vectorise the code because it is an array of particles instead of individual arrays of values. To confirm this add the `-fopt-info-vec` flag to the compilation in `Makefile`. What do you notice about the output if you recompile?

We will now change the vector of particles to individual vectors. You do this by changing the following code:

```
1 struct particle {
2     float x, y, z;        // position
3     float vx, vy, vz;     // velocity
4     float ax, ay, az;     // acceleration
5 };
6 typedef vector<particle> particles;
```

to (notice that we change the name of the structure from `particle` to `particles`):

```
1 struct particles {
2     vector<float> x, y, z;    // position
3     vector<float> vx, vy, vz; // velocity
4     vector<float> ax, ay, az; // acceleration
5 };
```

Now we need to make a few changes to the code. First, we have to change everywhere we reference a field so for example, `plist[j].x` becomes `plist.x[j]`.

We also need to deal with allocating space for the lists. It turns out that the `push_back` method is not every efficient because it checks the size of the vector each time. We would

also be pushing elements onto `x`, `y` and `z` but need to resize the other field vectors as well. The solution is to not use `clear`, `reserve` or `push_back` at all. Instead we will use `resize` on each of the 9 vectors which allocates and initialises all of the elements. We then simply use indexing (`[]`) to read or set values.

Your code should compile and run. Did you notice any change in the vectorisation report? Time the code. Is this what you expected?

Exercise 4 [Eliminate Divergence]

Consider this part of the code.

```
1 for(int j=0; j<n; ++j) { // Depends on all other particles
2     if (i==j) continue; // Skip self interaction
```

We learned weeks ago about divergence and how it is really bad for vectorisation. The loop cannot be vectorised because of the `if` statement. We cannot just remove it, otherwise we would divide by zero for every particle. Sometimes equations can be rewritten to avoid problematic cases, and there are other strategies to eliminate branches from inside loops. For this problem there is a simple solution.

- remove the `if` statement: `if (i==j) continue;`
- duplicate the `j` loop.
- on the first `j` loop, iterate from 0 to `j<i`
- on the second `j` loop, iterate from `i+1` to `j<n`

In this way you avoid `i`. Now if you compile you should notice a change in the vector report. Time the code.

Exercise 5 [OpenMP]

Convert your code to OpenMP by parallalising over the `i` loop. Don't forget to compile with `-fopenmp`. Are there any data dependencies between the `i` loop iterations that would require synchronisation (e.g., atomic updates)? How long does the code take now? On Eiger change `N` from 50'000 to 500'000. If you time it on Eiger with 128 cores (256 vcores) how long does it take? The `time` command report **real** and **user** times. What are those two numbers? Explain what they mean.